

Mini Exercise — Từ API Test đến Contract Breaking Change

Phiên bản: 1.0

Thời lượng: 90 phút tại lớp

Hình thức: Cá nhân; được trao đổi theo cặp, mỗi sinh viên nộp minh chứng riêng

Đối tượng kiểm thử: Product Service — `GET /product/:id`

Mục lục

- [1. Mục tiêu học tập](#)
- [2. Bối cảnh và phạm vi](#)
- [3. Chuẩn bị trước giờ học](#)
- [4. Kịch bản 90 phút](#)
- [5. Phần A — AI-assisted test design](#)
- [6. Phần B — Postman và Newman](#)
- [7. Phần C — Pact và breaking change](#)
- [8. Exit ticket và bài nộp](#)
- [9. Tiêu chí đánh giá](#)
- [10. Hướng dẫn cho giảng viên và trợ giảng](#)
- [11. Đáp án kỳ vọng](#)
- [12. Tài liệu tham khảo](#)

1. Mục tiêu học tập

Sau bài thực hành, sinh viên có thể:

1. Dùng AI để đề xuất test case cho một API, sau đó tự đánh giá và sửa kết quả AI.
2. Chuyển test case đã duyệt thành dữ liệu chạy lặp trong Postman.
3. Chạy collection bằng Newman và đọc kết quả assertion từ báo cáo JSON.
4. Giải thích vai trò khác nhau của functional API test và consumer-driven contract test.
5. Chứng minh một thay đổi response có thể vẫn chạy được về mặt HTTP nhưng phá vỡ contract của consumer.

Bài tập rút gọn pipeline của HW06 từ ba API và khoảng 10 giờ xuống một API trong 90 phút. Các phần CI/CD, GitHub Issues, báo cáo PDF riêng, Agent Skill hoàn chỉnh và AI Critique 200–300 từ không thuộc phạm vi bài này.

2. Bối cảnh và phạm vi

Frontend đang sử dụng Product Service và kỳ vọng endpoint sau:

```
GET /product/{id}
Authorization: Bearer <ISO-8601 timestamp trong vòng 1 giờ>
X-Student-Id: <MSSV>
```

Response thành công phải có bốn field:

```
{
  "id": "10",
  "type": "CREDIT_CARD",
  "name": "28 Degrees",
  "version": "v1"
}
```

Các hành vi đã biết:

- `id` tồn tại và token hợp lệ: `200 OK` .
- `id` không tồn tại và token hợp lệ: `404 Not Found` , body có field `message` .
- Thiếu token hoặc token hết hạn: `401 Unauthorized` , body có field `error` .

Phạm vi chỉ gồm `GET /product/:id` . Sinh viên không cần chạy CRUD đầy đủ, Pact Broker, Docker hoặc CI/CD.

3. Chuẩn bị trước giờ học

3.1. Sinh viên

- Node.js 18 hoặc 20 LTS, npm và Git.
- Postman Desktop hoặc Postman Web kèm Desktop Agent.
- Newman đã cài và có thể gọi bằng lệnh `newman --version`.
- Repository đã được clone và dependencies đã được cài trước buổi học.
- Một công cụ AI có thể lưu lại prompt và output.

Từ thư mục gốc repository, kiểm tra nhanh:

```
node --version
npm --version
newman --version
```

Nếu chưa cài dependencies, thực hiện trước buổi học:

```
npm ci --prefix src/sample-api/pact-workshop-js
npm install --global newman
```

3.2. Tập sẽ sử dụng

Mục đích	Đường dẫn
Collection	<code>src/postman/collections/product-service-data-driven.postman_collection.json</code>
Environment	<code>src/postman/environments/local.postman_environment.json</code>
Data mẫu	<code>src/postman/data/get-product-by-id.data.json</code>
Consumer Pact test	<code>src/sample-api/pact-workshop-js/consumer/src/api.pact.spec.js</code>
Provider verifier	<code>src/sample-api/pact-workshop-js/provider/product/product.pact.test.js</code>

Mục đích	Đường dẫn
Provider controller	<code>src/sample-api/pact-workshop- js/provider/product/product.controller.js</code>

4. Kịch bản 90 phút

Thời gian	Hoạt động	Checkpoint
0–10 phút	Khởi động provider, health check, import Postman assets	<code>GET /health</code> trả <code>200</code>
10–25 phút	AI đề xuất test case; sinh viên audit và chọn phạm vi	Bảng audit có ít nhất 6 dòng
25–45 phút	Tạo iteration data, thêm MSSV header và một assertion	Chạy Postman không có assertion fail
45–55 phút	Chạy Newman, xuất JSON report	Newman exit code <code>0</code>
55–65 phút	Sinh Pact và verify provider phiên bản hiện tại	Consumer và provider đều pass
65–77 phút	Tạo breaking change và verify lại	Provider verification fail đúng lý do
77–84 phút	Khôi phục provider và verify lại	Provider verification pass trở lại
84–90 phút	Exit ticket, đóng gói minh chứng	Đủ bốn deliverable

Nếu trễ checkpoint quá 5 phút, dùng tệp data mẫu hiện có và chuyển ngay sang bước tiếp theo. Mục tiêu là quan sát trọn chu trình, không phải viết số lượng test case lớn.

5. Phần A — AI-assisted test design

Thời lượng: 15 phút — 2 điểm

Bước A1 — Gửi prompt có ràng buộc

Gửi cho AI contract ở phần 2 và yêu cầu đề xuất đúng **6 test case** bao phủ:

- happy path;
- partition của `id` ;
- authentication;
- response schema.

Prompt phải yêu cầu AI trả về các cột: `tc_id` , input, expected status, expected fields và rationale. Không dùng prompt kiểu “generate all tests”.

Bước A2 — Human review

Audit cả 6 test case bằng bảng sau:

TC	Nhãn	Nhận xét hoặc chỉnh sửa
AI-01	<code>VALID</code> , <code>INVALID</code> hoặc <code>INCOMPLETE</code>	...

Quy tắc:

- Gắn nhãn cho mọi test case.
- Sửa ít nhất một test case `INVALID` hoặc `INCOMPLETE` . Nếu cả 6 đều hợp lệ, chỉ ra một giả định mà AI chưa nêu rõ và bổ sung nó.
- Chọn 5 test case để thực thi, trong đó bắt buộc có ít nhất một `200` , một `404` và một `401` .
- Tự bổ sung một test idea AI đã bỏ sót và ghi một câu giải thích. Test idea này có thể thay thế một trong 5 case được chọn.

Không gửi source code riêng tư, token thật hoặc dữ liệu cá nhân cho AI. Chỉ dùng contract và dữ liệu giả lập của bài.

6. Phần B — Postman và Newman

Thời lượng: 30 phút — 3 điểm

Bước B1 — Khởi động provider

Mở terminal thứ nhất:

```
npm start --prefix src/sample-api/pact-workshop-js/provider
```

Kiểm tra tại terminal thứ hai:

```
curl http://localhost:8080/health
```

Kết quả mong đợi: `{"status": "ok"}`.

Bước B2 — Tạo iteration data

Sao chép `src/postman/data/get-product-by-id.data.json` thành `mini-get-product.data.json`, sau đó giữ đúng 5 dòng tương ứng với các case đã duyệt ở phần A. Không thay đổi tên các key mà collection đang sử dụng.

Mỗi dòng tối thiểu phải có:

```
{
  "tc_id": "MINI_01",
  "description": "Existing product with valid token",
  "product_id": "10",
  "auth_header": "{{validToken}}",
  "expected_status": 200,
  "expect_field_id": "10",
  "expect_field_type": "CREDIT_CARD",
  "expect_field_name": "28 Degrees"
}
```

Với case lỗi, dùng `expect_message_field: "message"` cho `404` hoặc `expect_error_field: "error"` cho `401`.

Bước B3 — Thêm dấu vết cá nhân và assertion

Import collection và environment ở phần 3.2 vào Postman. Thêm environment variable:

```
studentId = <MSSV của bạn>
```

Trong pre-request script của request `GET /product/:id [by id]`, thêm:

```
pm.request.headers.upsert({  
  key: "X-Student-Id",  
  value: pm.environment.get("studentId")  
});
```

Trong test script của cùng request, tự viết thêm **một assertion** kiểm tra `Content-Type` hoặc response time. Ví dụ assertion chỉ dùng để tham khảo cấu trúc:

```
pm.test("[MINI] Response is JSON", () => {  
  pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");  
});
```

Chạy folder `GET – Happy Path` bằng Collection Runner với `mini-get-product.data.json`. Collection này dùng assertion dựa trên `expected_status`, nên 5 iteration có thể bao gồm cả positive và negative case.

Bước B4 — Chạy Newman

Export collection và environment sau khi chỉnh sửa thành:

```
mini-product-service.postman_collection.json  
mini-local.postman_environment.json
```

Chạy:

```
newman run mini-product-service.postman_collection.json \  
  --environment mini-local.postman_environment.json \  
  --iteration-data mini-get-product.data.json \  
  --folder "GET – Happy Path" \  
  --reporter cli
```

```
--reporters cli,json \  
--reporter-json-export mini-newman-report.json
```

Checkpoint:

- Có đúng 5 iteration của request `GET /product/:id`.
- Không có assertion fail.
- `mini-newman-report.json` tồn tại.
- Console hoặc Postman Console cho thấy request có `X-Student-Id` đúng MSSV.

7. Phần C — Pact và breaking change

Thời lượng: 29 phút — 4 điểm

Bước C1 — Sinh consumer contract

Từ thư mục gốc repository:

```
npm run test:pact --prefix src/sample-api/pact-workshop-js/consumer -- \
  --testNamePattern="GET /product/:id"
```

Kiểm tra file được sinh:

```
src/sample-api/pact-workshop-js/consumer/pacts/FrontendWebsite-ProductService.json
```

Mở Pact JSON và tìm interaction “a request for product 10”. Ghi lại bốn field mà consumer yêu cầu trong response body.

Bước C2 — Verify provider phiên bản hiện tại

Dừng provider đang chạy ở phần B để giải phóng port **8080**, sau đó chạy:

```
npm run test:pact --prefix src/sample-api/pact-workshop-js/provider
```

Kết quả mong đợi: provider verification pass.

Bước C3 — Cố ý tạo breaking change

Trong **product.controller.js**, chỉ tại hàm **getById**, tạm đổi response thành cấu trúc dùng **title** thay cho **name**:

```
exports.getById = async (req, res) => {
  const product = await repository.getById(req.params.id);
  product
    ? res.send({
        id: product.id,
        type: product.type,
```

```
        title: product.name,  
        version: product.version  
    })  
    : res.status(404).send({ message: "Product not found" });  
};
```

Chạy lại provider verification:

```
npm run test:pact --prefix src/sample-api/pact-workshop-js/provider
```

Chụp phần lỗi cho thấy response thực tế thiếu `name` hoặc không khớp body mà consumer yêu cầu. Một lỗi có chủ đích ở bước này là kết quả đúng.

Bước C4 — Khôi phục và xác nhận

Khôi phục đúng dòng xử lý ban đầu:

```
product  
  ? res.send(product)  
  : res.status(404).send({ message: "Product not found" });
```

Chạy provider verification lần cuối. Bài chỉ hoàn thành khi verification pass trở lại và source code không còn breaking change.

8. Exit ticket và bài nộp

Thời lượng: 6 phút — 1 điểm

Trả lời ngắn, tối đa 120 từ cho cả hai câu:

1. Vì sao Postman/Newman test và Pact test không thay thế lẫn nhau?
2. Tại sao đổi `name` thành `title` là breaking change dù HTTP status vẫn có thể là `200` ?

Nộp một file `.zip` tên `<MSSV>_Mini_API_Contract.zip` gồm:

1. `test-design.md` : prompt, AI output rút gọn, bảng audit, test idea tự bổ sung và exit ticket.
2. `mini-get-product.data.json` .
3. `mini-product-service.postman_collection.json` và `mini-local.postman_environment.json` .
4. `mini-newman-report.json` .
5. Hai ảnh: `pact-fail.png` và `pact-pass-restored.png` .

Không nộp `node_modules` , token, cookie, API key hoặc credential.

9. Tiêu chí đánh giá

Tiêu chí	Điểm	Điều kiện đạt tối đa
AI-assisted design và human audit	2.0	Đủ 6 case, audit có lý do, có sửa/bổ sung bằng đánh giá của người học
Data-driven Postman test	2.0	5 iteration gồm 200 , 404 , 401 ; có MSSV header và assertion tự viết
Newman execution	1.0	Exit code 0 , báo cáo JSON hợp lệ, không có assertion fail
Consumer Pact và provider verification	1.0	Sinh đúng Pact và verify baseline pass
Breaking change experiment	3.0	Có fail đúng nguyên nhân, khôi phục source và verify pass lại
Exit ticket và vệ sinh bài nộp	1.0	Giải thích đúng, đủ file, không lộ secret
Tổng	10.0	

Điểm tối thiểu để hoàn thành: **6/10**, đồng thời bắt buộc có Newman report và ít nhất một lần provider verification.

10. Hướng dẫn cho giảng viên và trợ giảng

10.1. Chuẩn bị trước lớp

1. Clone repository trên máy mẫu và chạy `npm ci --prefix src/sample-api/pact-workshop-js`.
2. Cài Newman, chạy thử provider, collection và cả hai Pact command.
3. Bảo đảm port `8080` không bị ứng dụng khác chiếm.
4. Chuẩn bị sẵn Pact JSON và Newman JSON mẫu để cấp cho sinh viên gặp lỗi cài đặt sau phút 10.
5. Nhắc sinh viên không chạy provider thủ công đồng thời với provider verifier, vì verifier tự mở server trên port `8080`.

10.2. Can thiệp theo checkpoint

- **Phút 10:** máy chưa chạy được provider chuyển sang ghép cặp hoặc dùng máy dự phòng.
- **Phút 25:** sinh viên chưa có bảng audit dùng 6 case mẫu ở phần 11, nhưng vẫn phải tự gắn nhãn và giải thích.
- **Phút 45:** sinh viên chưa tạo xong data file dùng tệp gốc và chọn 5 dòng.
- **Phút 65:** nếu consumer Pact test không chạy do môi trường, cấp Pact JSON mẫu để tiếp tục provider verification.
- **Phút 77:** kiểm tra sinh viên hiểu fail là có chủ đích; không dành thời gian “sửa test để pass”.
- **Phút 84:** yêu cầu mọi sinh viên khôi phục `product.controller.js` trước khi nộp.

Không chấm lỗi cài đặt như lỗi kiến thức nếu sinh viên vẫn hoàn thành phần phân tích bằng artifact dự phòng.

11. Đáp án kỳ vọng

11.1. Bộ test case tối thiểu hợp lý

TC	Input chính	Kết quả kỳ vọng	Kỹ thuật
01	<code>id=10</code> , token hợp lệ	<code>200</code> ; đủ <code>id</code> , <code>type</code> , <code>name</code> , <code>version</code>	Happy path + schema
02	<code>id=99</code> , token hợp lệ	<code>404</code> ; có <code>message</code>	Non-existing partition
03	<code>id=abc</code> , token hợp lệ	<code>404</code> ; có <code>message</code>	Invalid-format partition theo hành vi hiện tại
04	<code>id=10</code> , thiếu token	<code>401</code> ; có <code>error</code>	Authentication
05	<code>id=10</code> , token cũ	<code>401</code> ; có <code>error</code>	Authentication boundary
06	<code>id=10</code> , token hợp lệ	<code>Content-Type</code> là JSON	Response metadata

`id=abc` trả `404` là hành vi của SUT hiện tại, không nên tự suy đoán thành `400` nếu specification không quy định validation số. Đây là một ví dụ tốt để audit đề xuất của AI.

11.2. Kết luận breaking change

Consumer contract yêu cầu field `name` . Provider đổi field đó thành `title` khiến shape của response không còn thỏa Pact, dù endpoint vẫn trả `200` . Provider verification phải fail; khôi phục `name` phải làm verification pass lại.

Functional test trả lời “API có thực hiện đúng các case và assertion ta chạy không?”. Pact provider verification trả lời “provider có còn đáp ứng đúng những interaction mà consumer đã công bố không?”. Hai lớp kiểm thử có mục tiêu và phạm vi khác nhau.

12. Tài liệu tham khảo

- HW06 – API Testing: `docs/reports/2026.HW06.API Testing_En (1).md` .
- Newman command-line options: <https://github.com/postmanlabs/newman#command-line-options>.
- Pact — 5-minute getting started guide: <https://docs.pact.io/5-minute-getting-started-guide>.
- Pact — provider verification:
https://docs.pact.io/implementation_guides/javascript/docs/provider.
- Postman collection và data files của seminar: `src/postman/README.md` .
- Pact assets của seminar: `src/pact/README.md` .